



Module – 2

- **Strings:** Working with strings as single things, working with the parts of a string, Length, Traversal and the for loop, Slices, String comparison, Strings are immutable, the in and not in operators, A find function, Looping and counting, Optional parameters, The built-in find method, The split method, Cleaning up your strings, The string format method.
- **Tuples:** Tuples are used for grouping data, Tuple assignment, Tuples as return values, Composability of Data Structures.
- **Lists:** List values, accessing elements, List length, List membership, List operations, List slices, Lists are mutable, List deletion, Objects and references, Aliasing, cloning lists, Lists and for loops, List parameters, List methods, Pure functions and modifiers, Functions that produce lists, Strings and lists, list and range, Nested lists, Matrices.
- **Chapter: 5.1, 5.2, 5.3**
- **CO2:** Demonstrate proficiency in manipulating and processing strings, lists, and tuples for data handling and problem-solving.

Function example

```
def factorial(n):  
    if n == 0 or n == 1:  
        return 1  
    else:  
        return n * factorial(n - 1)  
  
num = int(input("Enter a number: "))  
print("Factorial of", num, "is", factorial(num))
```

Strings

- **What is a Compound Data Type?**
- A **compound data type** is one that is **made up of smaller pieces** — it can hold multiple values or elements inside it.
- In Python, examples include:
- **String (str)** → made up of characters
- **List (list)** → made up of elements (which can be of any type)
- **Tuple / Pair (tuple)** → made up of a fixed number of elements
- **Dictionary (dict)** → made up of key–value pairs

Strings Are Objects

- In Python, **everything is an object** — including strings. This means every string has:
- **attributes** (data stored in the object)
- **methods** (functions that belong to that object)
- You can use a **dot (.)** after the string to call a **method**
- Example:

```
our_string = "Hello, World!"  
all_caps = our_string.upper()  
print(all_caps)
```

- Output:
- HELLO, WORLD!

Common String Methods

Method	Description	Example	Output
.upper()	Converts to uppercase	"hello".upper()	"HELLO"
.lower()	Converts to lowercase	"HELLO".lower()	"hello"
.capitalize()	Makes first letter uppercase	"hello".capitalize()	"Hello"
.swapcase()	Switches upper ↔ lower	"HeLLo".swapcase()	"hEllo"
.title()	Capitalizes every word	"hello world".title()	"Hello World"
.strip()	Removes spaces from ends	" hi ".strip()	"hi"

Indexing

- Subscript operator [] is used to access the individual characters in a string.
- The value or the expression inside the [] brackets is called **index**.

	P	Y	T	H	O	N
Index →	0	1	2	3	4	5

Visualizing Indices with enumerate()

You can use the enumerate() function to see both **indices** and **characters**:

```
fruit = "banana"
```

```
print(list(enumerate(fruit)))
```

- **Output:**

- [(0, 'b'), (1, 'a'), (2, 'n'), (3, 'a'), (4, 'n'), (5, 'a')]

Indexing Works the Same for Lists

- You can use the same [] notation to access elements of a **list**.
- Example:

```
prime_numbers = [2, 3, 5, 7, 11, 13]
```

```
print(prime_numbers[4])
```

- **Output:**

- 11

- **Indexing Works with strings:**

```
friends = ["Joe", "Zoe", "Brad", "Angelina", "Zuki"]
```

```
print(friends[3])
```

- **Output:**

- Angelina

length of a string

- 1. The len() Function

- The built-in function len() tells you how many **characters** are in a string.

- Example:

```
word = "banana"
```

```
print(len(word))
```

- **Output: 6**

- 2. Getting the Last Character (Common Mistake!)

```
size = len(word)
```

```
last = word[size]
```

```
# ❌ WRONG!
```

- But this gives an **error**:

- IndexError: string index out of range

Correct way:

```
size = len(word)
```

```
last = word[size - 1]
```

```
print(last)
```


3. Negative Indices

- Python gives an easier and shorter way to get characters from the end **negative indexing**.

Index	-6	-5	-4	-3	-2	-1
Letter	b	a	n	a	n	a

- Examples:
- `word = "banana"`
- `print(word[-1])` # last letter
- `print(word[-2])` # second to last
- `print(word[-3])` # third to last
- .

Negative indexing Works for Lists Too!

- Negative indexing and len() work in the same way with **lists**.
- Example:

```
numbers = [10, 20, 30, 40, 50]
```

```
print(len(numbers))  # 5
```

```
print(numbers[-1])   # 50 (last element)
```

```
print(numbers[-2])   # 40
```

Traversal and the for loop

- **Traversal** is a processing every element of a sequence.
- Traversal = start at one end of a sequence (string, list, etc.) and process each element in turn.

```
fruit = "banana"
```

```
ix = 0
```

```
while ix < len(fruit):
```

```
    letter = fruit[ix]
```

```
    print(letter)
```

```
    ix += 1
```

- **Output:**

b

a

n

a

n

a

This works, but it's long and error-prone here, ix needs to manage and to stop.

for loop for tracing

```
word = "Banana"
```

```
for letter in word:
```

```
    print(letter)
```

- Each iteration assigns the next character to letter and better way.

abecedarian duckling names (fixed)

- In the children's book *Make Way for Ducklings* by **Robert McCloskey**, the ducklings are named:
- **Jack, Kack, Lack, Mack, Nack, Ouack, Pack, Quack**
- Notice that most names follow a **pattern**:
- Start with a letter (J, K, L, M, N, O, P, Q)
- End with “**ack**”
- Only **O** and **Q** are exceptions — they need “**u**” before “**ack**”.

prefixes = "JKLMNOPQ"

suffix = "ack"

for p in prefixes:

 print(p + suffix)

Output:

Jack

Kack

Lack

Mack

Nack

Oack

Pack

Quack

Slices

- A substring of a string is obtained by taking a slice. Similarly, we can slice a list to refer to some sublist of the items in the list:

```
phrase = "Pirates of the Caribbean"
```

```
print(phrase[0:7])
```

- Pirates

```
print(phrase[11:14])
```

- the

```
print(phrase[13:24])
```

- e Caribbean

```
friends = ["Joe", "Zoe", "Brad", "Angelina", "Zuki", "Thandi", "Paris"]
```

```
print(friends[2:4])
```

- ['Brad', 'Angelina']

Slice...

- Three tricks with :
- If you omit the first index (before the colon), the slice starts at the beginning of the string (or list).
- If you omit the second index, the slice extends to the end of the string (or list).
- If you provide value for n that is bigger than the length of the string (or list), the slice will take all the values up to the end. (It won't give an “out of range” error like the normal indexing operation does.)

```
word = "banana"
```

```
word[:3]
```

```
'ban'
```

```
word[3:]
```

```
'ana'
```

```
word[3:999]
```

```
'ana'
```

String comparison

- You can compare strings directly using the **comparison operators** you already know.

- **Example 1 — Equality**

```
word = "banana"
```

```
if word == "banana":
```

```
    print("Yes, we have no bananas!")
```

- **Output:**

Yes, we have no bananas!

Operator	Meaning
==	equal to
!=	not equal to
<	comes before
	comes after
<=	before or same
=	after or same

Lexicographical (Alphabetical) Order

- Strings are compared **character by character**, using the **Unicode** (ASCII) order of each letter.

- Example:

```
word = "apple"
```

```
if word < "banana":
```

```
    print("Your word,", word, ", comes before banana.")
```

```
elif word > "banana":
```

```
    print("Your word,", word, ", comes after banana.")
```

```
else:
```

```
    print("Yes, we have no bananas!")
```

- Output:

Your word, apple, comes before banana.

Case Sensitivity (Important!)

- In Unicode:
- All **uppercase letters** come **before** all lowercase letters. Ex: **"Zebra" < "banana"**
- To make comparisons *case-insensitive*, convert both strings to the **same case** first usually lowercase.
- Example:

```
word = "Zebra"
if word.lower() < "banana":
    print("Your word,", word, ", comes before banana.")
elif word.lower() > "banana":
    print("Your word,", word, ", comes after banana.")
else:
    print("Yes, we have no bananas!")
```

- Output:

Your word, Zebra, comes after banana.

Strings are immutable

- “Immutable” means **unchangeable** once a string is created, **it cannot be modified**.
- You can’t change its characters in place; you can only **create a new string** based on it.
- **Example (What *not* to do):**

```
greeting = "Hello, world!"
```

```
greeting[0] = 'J' # This will cause an error
```

```
print(greeting)
```

- **Error:**
- `TypeError: 'str' object does not support item assignment`

Correct Way — Create a New String

- If you want to change part of a string, you must **build a new one** using slicing and concatenation.

```
greeting = "Hello, world!"
```

```
new_greeting = "J" + greeting[1:]
```

```
print(new_greeting)
```

- **Output:**

Jello, world!

- **Why Are Strings Immutable?**

- **Efficiency:**

Python can store and reuse strings safely since they never change.

- **Safety:**

Immutable objects are easier to reason about in programs they won't change accidentally.

- **Hashing:**

Immutable objects (like strings) can be used as **dictionary keys** or **set elements** because their value won't change.

Membership operators

- **in** Returns True if a sequence with the specified value is present in the object `y in x`

```
x = ["apple", "banana"]
```

```
print("banana" in x)
```

- # returns True because a sequence with the value "banana" is in the list

- **not in** Returns True if a sequence with the specified value is not present in the object `x not in y`

```
x = ["apple", "banana"]
```

```
print("pineapple" not in x)
```

- # returns True because a sequence with the value "pineapple" is not in the list

Some more examples

- **"p" in "apple"** Output: True
- **"i" in "apple"** Output: False
- **"ap" in "apple"** Output: True
- **"pa" in "apple"** Output: False
- **"a" in "a"** Output: True
- **"apple" in "apple"** Output: True
- **"" in "a"** Output: True
- **"" in "apple"** Output: True

- **"x" not in "apple"** Output: True

write a function that removes all the vowels from a string:

- Combine the **in** operator with string concatenation using **+**, to remove the vowels
- #Function:

```
def remove_vowels(phrase):
```

```
    vowels = "aeiou"
```

```
    string_sans_vowels = ""
```

```
    for letter in phrase:
```

```
        if letter.lower() not in vowels:
```

```
            string_sans_vowels += letter
```

```
    return string_sans_vowels
```

```
#main program
```

```
text = 'Hi i am the program explaining the vowels. This function removes vowels'
```

```
print ('The text before removing vowels', text)
```

```
print ('The text after removing vowels', remove_vowels(text))
```

Output

- The text before removing vowels `Hi i am the program explaining the vowels.`
This function removes vowels
- The text after removing vowels `H m th prgrm xplnng th vwls. Ths fnctn rmvs vwls`

To count number of occurrence of vowels

```
def count_vowels():  
    text = "Programming is easy".lower()  
    vowels = "aeiou" # list of vowels count = 0  
    count = 0  
    # check each character  
    for char in text:  
        if char in vowels:  
            count += 1  
  
    print("The number of vowels in \n", text, "\n is:", count)  
  
# call the function  
count_vowels()
```

Printing the Tables

```
def tables(n):  
    print("i \tx", n)  
    for i in range(1, 11):  
        print(i, "\t", i*n)  
    return  
  
# main program  
for i in range( 2, 6):  
    tables(i)
```

i	ix 2	i	ix 3	i	ix
1	2	1	3	5	
2	4	2	6	1	5
3	6	3	9	2	10
4	8	4	12	3	15
5	10	5	15	4	20
6	12	6	18	5	25
7	14	7	21	6	30
8	16	8	24	7	35
9	18	9	27	8	40
10	20	10	30	9	45
				10	50

Finding the power

```
print("i\ti**2\ti**3\ti**5\ti**10\ti**20")
```

```
for i in range(1, 11):
```

```
    print(i, "\t", i**2, "\t", i**3, "\t", i**5, "\t", i**10, "\t", i**20)
```

Output:

i	i**2	i**3	i**5	i**10	i**20
1	1	1	1	1	1
2	4	8	32	1024	1048576
3	9	27	243	59049	3486784401
4	16	64	1024	1048576	1099511627776
5	25	125	3125	9765625	95367431640625
6	36	216	7776	60466176	3656158440062976
7	49	343	16807	282475249	79792266297612001
8	64	512	32768	1073741824	1152921504606846976
9	81	729	59049	3486784401	12157665459056928801
10	100	1000	100000	10000000000	100000000000000000000

Sequences

- Series of items that are often related
- 3 types of sequences (sequential Data Types)
- Strings
- Lists
- Tuples



Python Lists

- A **list** is a sequence of data values called *items* or *elements*.
- The items in a list need not be of the same type.
- It is a **mutable** data type.
- **Creating a List**
- A list is defined as a collection of comma-separated values (items) enclosed in **square brackets []**.
- A list can also be created using the **list()** function.

```
List2 = ['Physics', 'Chemistry', 10, 30, 20.5]
```

```
List3 = [[1, 2], 4, 5]
```

```
List4 = []    # Creating an Empty List
```

```
List5 = list(string / tuple / dict / iterable)
```

Creating the list using list

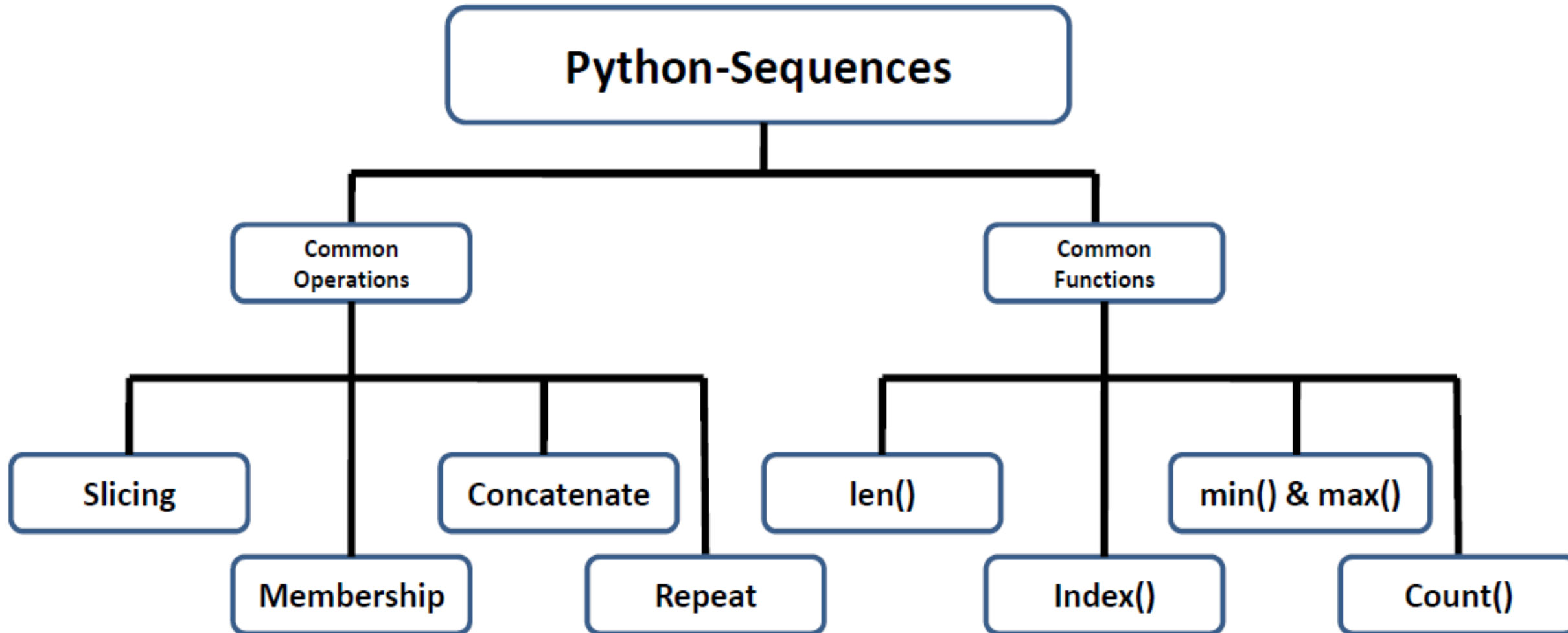
- `list(range(4))` # [0, 1, 2, 3]
- `list(range(1, 4))` # [1, 2, 3]
- `list(range(2, 10, 2))` # [2, 4, 6, 8]
- `list(range(-1, -11, -2))` # [-1, -3, -5, -7, -9]



Lists: Accessing Elements

- Each item in a list has a **unique index** that specifies its position.
- The list index can be either **positive** or **negative**.
- **Positive index** starts from the **left** of the list and ranges from **0** to *(length of the list – 1)*.
- **Negative index** starts from the **right** of the list and ranges from **-1** to *–(length of the list)*.
- List2 = [[1, 2], 4, 5]
- # Accessing Elements
- List2[0] # [1, 2]
- List2[1] # 4
- List2[2] # 5
- List2[0][0] # 1
- List2[0][1] # 2

Lists: Operations & Functions



List Slices

- A **slice** is a list consisting of selected elements from an existing list.
- **Slicing** uses three index values: **low**, **high**, and **step_value**.
- **Syntax:**
- List1[low : high : step_value]

```
List1 = ['a', 'b', 'c', 'd', 'e', 'f']
```

```
List1[1:3]    # ['b', 'c']
```

```
List1[:4]     # ['a', 'b', 'c', 'd']
```

```
List1[3:]     # ['d', 'e', 'f']
```

```
List1[:]      # ['a', 'b', 'c', 'd', 'e', 'f']
```

```
List1[::2]    # ['a', 'c', 'e']
```

```
List1[::-1]   # ['f', 'e', 'd', 'c', 'b', 'a']
```

Lists: Methods

- `a = [1, 2, 3, 4, 5]; b = [10, 11, 12, 13]; c = [20, 1, -2, 13]`
- **1. reverse() Syntax:** `A.reverse()`
- **Description:** Reverses the elements of the list *in place* (i.e., it modifies the original list).
- **Example:**

`a.reverse()`

`print(a)` # Output: `[5, 4, 3, 2, 1]`

- **2. sort() Syntax:** `A.sort()`
- **Description:** Sorts the elements of the list *in place* in **ascending order** by default.
- **Example:**

`c.sort()`

`print(c)` # Output: `[-2, 1, 13, 20]`

- **3. extend()**
- **Syntax:** A.extend(iterable)
- **Description:** Extends the list by appending all elements from another iterable (such as another list).
- **Example:**

```
a = [1, 2, 3, 4, 5]
```

```
b = [10, 11, 12, 13]
```

```
a.extend(b)
```

```
print(a)    # Output: [1, 2, 3, 4, 5, 10, 11, 12, 13]
```

- **4. clear()**
- **Syntax:** A.clear()
- **Description:** Removes **all elements** from the list A.
- **Example:**

```
A = [1, 2, 3, 4]
```

```
A.clear()
```

```
print(A)    # Output: []
```

-

5. copy()

- **Syntax:** A.copy()
- **Description:** Returns a **shallow copy** of the list A.
- **Example:**

```
A = [1, 2, 3, 4]
```

```
B = A.copy()
```

```
print(B)    # Output: [1, 2, 3, 4]
```

- **6. count()**
- **Syntax:** A.count(obj)
- **Description:** Returns the **number of times** the element obj occurs in the list A.
- **Example:**

A = [1, 2, 2, 3, 2, 4]

print(A.count(2)) # Output: 3

- **7. index()**
- **Syntax:** A.index(obj)
- **Description:** Returns the **lowest index** in the list A where the element obj appears **first**.
- **Example:**

A = [10, 20, 30, 20, 40]

print(A.index(20)) # Output: 1

- **8 insert()**
- **Syntax:** A.insert(index, obj) : Inserts the object obj at the specified **index** in the list.

- **Example:**

```
A = [10, 20, 30, 40]
```

```
A.insert(2, 25)
```

```
print(A)    # Output: [10, 20, 25, 30, 40]
```

- **9. append()**
- **Syntax:** A.append(obj) : Adds the object obj to the **end** of the list.
- **Example:**

```
A = [1, 2, 3]
```

```
A.append(4)
```

```
print(A)    # Output: [1, 2, 3, 4]
```

- **10. remove()**
- **Syntax:** A.remove(obj) : Removes the **first occurrence** of the element obj from the list.

- **Example:**

A = [5, 10, 15, 10, 20]

A.remove(10)

print(A) # Output: [5, 15, 10, 20]

Python Tuple

- A **tuple** is a sequence of Python objects separated by **commas**.
- A tuple is similar to a list in terms of **indexing**, **nesting**, and **repetition**, but it is **immutable**, unlike lists which are **mutable**.
- Tuples are often used to store **fixed collections** of items.
- **Creating Tuples**
- A tuple is created by enclosing elements in **parentheses** (optional).

- **Examples:**

```
tup1 = (111, 12, 11)
```

```
tup2 = 111, 12, 11 # Parentheses are optional
```

- **Creating an Empty Tuple**

```
tup4 = ()
```

```
tup5 = tuple()
```

- **Creating a Nested Tuple**

- A nested tuple is created by placing one tuple **inside another**.

```
tup6 = ((1, 2), 3, 4, 'John', 'Ahmad', 'Anil', 'Khan')
```


Tuples in Python

- A **tuple** is an **ordered collection** of elements, similar to a list.
- Unlike lists, **tuples are immutable** — once created, their elements **cannot be modified** (no adding, removing, or changing values).
- Tuples can store **different data types** such as integers, strings, floats, or even other tuples.

- **Creating Tuples**

- # Creating tuples

```
tup1 = (10, 20, 30)
```

```
tup2 = ('apple', 'banana', 'cherry')
```

```
tup3 = (10, 'John', 89.5)
```

```
print(tup1) # Output: (10, 20, 30)
```

```
print(tup2) # Output: ('apple', 'banana', 'cherry')
```

```
print(tup3) # Output: (10, 'John', 89.5)
```

Tuple Packing

- **Tuple packing** is the process of **assigning multiple values** to a single tuple variable in one step.
- Python automatically packs the values into a tuple.
- **Example:**
- # Tuple Packing

```
student = ('John', 21, 'B.Sc', 8.7)
```

```
print(student)
```

- Output: ('John', 21, 'B.Sc', 8.7)

Tuple Unpacking

- **Tuple unpacking** is the process of **extracting individual values** from a tuple into separate variables.
- The number of variables must match the number of elements in the tuple.

- **Example:**

- # Tuple Unpacking

```
student = ('John', 21, 'B.Sc', 8.7)
```

```
    name, age, course, cgpa = student
```

```
print("Name:", name)
```

```
print("Age:", age)
```

```
print("Course:", course)
```

```
print("CGPA:", cgpa)
```

- **Output:**

- Name: John
- Age: 21
- Course: B.Sc
- CGPA: 8.7

Differences between Tuples and Lists		
Feature	List	Tuple
Mutability	Lists are mutable — elements can be changed, added, or removed after creation.	Tuples are immutable — once created, their elements cannot be changed.
Syntax	Created using square brackets [].	Created using parentheses ().
Performance	Slightly slower because of mutability (more memory overhead).	Slightly faster and use less memory .
Use case	Used when data needs to be modified or updated frequently.	Used when data is fixed or must not change (e.g., coordinates, constants).
Methods available	Lists have many built-in methods like append(), remove(), sort(), etc.	Tuples have very few methods , mainly count() and index().
Iteration	Iteration is a bit slower due to mutability.	Iteration is faster because of immutability.

Dictionary

- Dictionary is a built-in Python Data Structure and are used to store data values in key: value pairs. Each key is separated from its value by a colon (:).
- Dictionaries are not indexed by a sequence of numbers but indexed based on keys.
- The syntax for defining a dictionary is:
- **dictionary_name= {key_1: value_1, key_2: value2, key_3: value_3}**
- Or it can also be written as :
- **dictionary_name= { key_1: value_1,
 key_2: value_2,
 key_3: value_3,
 }**

The dictionary can also be create using dict() function.

Example

```
age = {'Alice': 25, 'Bob': 28}
```

```
age = dict({'Alice': 25, 'Bob': 28})
```

```
print(age)
```

Output: {'Alice': 25, 'Bob': 28}

```
age = {'Alice': 25, 'Bob': 28, 'Alice': 26}
```

```
print(age)
```

Output:

```
age
```

```
{'Alice': 28, 'Bob': 28}
```

Dictionary: Accessing Values

- To access values, a **dictionary** uses **keys**.
- The **key** is placed inside **square brackets ([])** — this returns the value corresponding to that key.
- The **get()** method can also be used to retrieve the value for a given key.
- **Example:**

```
age = {'Alice': 25, 'Bob': 28}
```

```
print(age['Alice']) # Output: 25
```

```
print(age.get('Bob')) # Output: 28
```

Dictionary: Displaying Contents

- **Example:**

```
age = {'Samar': 25, 'Sahil': 'twenty-two'}
```

- **items()** → Returns all key–value pairs in the dictionary.

```
age.items()
```

- # Output: dict_items([('Samar', 25), ('Sahil', 'twenty-two')])
- **keys()** → Returns all the keys in the dictionary.

```
age.keys()
```

- # Output: dict_keys(['Samar', 'Sahil'])
- **values()** → Returns all the values in the dictionary.

```
age.values()
```

- # Output: dict_values([25, 'twenty-two'])

Dictionaries Are Mutable

- A **dictionary** is a **mutable** data type — items can be **added** or **updated**.
- The **assignment operator (=)** can be used to **Add** a new item or to **Update** the value of an existing item
- If the key **already exists**, its **value is updated**.
- If the key **does not exist**, a **new key–value pair** is added.
- The **update()** method can also be used to add or modify multiple items at once.
- **Example:**

```
age = {'Alice': 25, 'Bob': 28}
```

- # Add or update single items

```
age['Alice'] = 26      # Updates existing key
```

```
age['Charlie'] = 30    # Adds new key–value pair
```

- # Add multiple items using update()

```
age.update({'Kavi': 29, 'Rahil': 23})
```

```
print(age)
```

- **Output:** {'Alice': 26, 'Bob': 28, 'Charlie': 30, 'Kavi': 29, 'Rahil': 23}

Dictionaries are **mutable**, so items can also be **removed**.

- **1. pop() Method**

- Removes a **specific item** using its key. Returns the **value** of the removed key.

```
a = {'Rahil': 26, 'Sahil': 22, 'Amar': 20}
```

```
a.pop('Rahil')
```

```
print(a)
```

- **Output:** {'Sahil': 22, 'Amar': 20}

2. clear() Method

- Removes **all items** from the dictionary, leaving it empty.

```
a.clear()
```

```
print(a)
```

- **Output:**

- {}

- **3. del Keyword**
- Can be used to:
 - Remove a **specific item**, or
 - **Delete the entire dictionary.**

```
a = {'Rahil': 26, 'Sahil': 22, 'Amar': 20}
```

```
del a['Sahil'] # Removes one key–value pair  
print(a)
```

- Output: {'Rahil': 26, 'Amar': 20}

```
del a # Deletes the dictionary completely
```

- **Note:**
After del a, the dictionary a no longer exists.

Common Dictionary Methods

- **1. len()**
- Returns the **number of items** (key–value pairs) in the dictionary.

```
A = {'Rahil': 26, 'Sahil': 22, 'Amar': 20}
```

```
print(len(A))
```

- **Output:**

- 3

- **2. sorted()**

- Returns a **new sorted list of keys** from the dictionary.

```
print(sorted(A))
```

- **Output:**

- ['Amar', 'Rahil', 'Sahil']

Common Dictionary Methods

- 3. `copy()`
- Returns a **shallow copy** of the dictionary.
- `B = A.copy()`
- `print(B)`
- **Output:** {'Rahil': 26, 'Sahil': 22, 'Amar': 20}

-

4. `get(key, d)`

- Returns the **value** for the given key.
- If the key does **not exist**, returns the **default value d** (defaults to None).

```
print(A.get('Sahil'))    # Output: 22
```

```
print(A.get('Kavi', 0))  # Output: 0
```

Iteration Through Dictionary Items

- We can **loop through key–value pairs** in a dictionary using the **items()** method.
- In each iteration:
 - The **first variable** represents the **key**.
 - The **second variable** represents the **corresponding value**.

- **Example:**

```
dict1 = {"Samar": 25, "Sahil": 29, "Rahil": 30, "Amar": 40}  
for key, value in dict1.items():  
    print(key, value)
```

- **Output:**

- Samar 25
- Sahil 29
- Rahil 30
- Amar 40

Iteration Through Keys / Values

- You can **loop through only the keys** or **only the values** of a dictionary using:
 - **keys()** → to access all keys
 - **values()** → to access all values
- **Example:**

```
dict1 = {"Samar": 25, "Sahil": 29, "Rahil": 30, "Amar": 40}
```

```
# Iterating through keys
```

```
for key in dict1.keys():  
    print(key)
```

- **Output:**

- Samar
- Sahil
- Rahil
- Amar

```
# Iterating through values  
for value in dict1.values():  
    print(value)
```

Output:

25
29
30
40

Counting Character Frequency in a String

- Program:

```
counts = dict()
```

```
text = "Counting Character Frequency in a String"
```

```
print("Counting...")
```

```
for letter in text:
```

```
    counts[letter] = counts.get(letter, 0) + 1
```

```
print("Counts:", counts)
```

Output:

Counting...

Counts: {'C': 2, 'o': 1, 'u': 2, 'n': 5, 't': 3, 'i': 3, 'g': 2, ' ': 5, 'h': 1, 'a': 3, 'r': 4, 'c': 2, 'e': 3, 'F': 1, 'q': 1, 'y': 1, 'S': 1}

An **empty dictionary** counts is created to store each character as a **key** and its **frequency** as the value.

- The loop iterates through each character in the input string.
- The method **get(letter, 0)** returns the current count of the character or 0 if it doesn't exist yet.
- Each time a character appears, its count is **incremented by 1**.

Program: Check if a Key Exists in a Dictionary

```
# Define a dictionary
```

```
students = {'Alice': 25, 'Bob': 28, 'Charlie': 22}
```

```
# Take input from user
```

```
key = input("Enter the name to check: ")
```

```
# Check if the key exists in the dictionary
```

```
if key in students:
```

```
    print("Yes", key, "exists in the dictionary with value", students[key])
```

```
else:
```

```
    print("No", key, "does not exist in the dictionary.")
```

Output: Enter the name to check: Bob

Yes, 'Bob' exists in the dictionary with value 28.

Program: Create an Empty Dictionary and Add Key–Value Pairs

```
# Create an empty dictionary
```

```
student = {}
```

```
# Add key-value pairs
```

```
student['Name'] = 'Alice'
```

```
student['Age'] = 21
```

```
student['Course'] = 'Computer Science'
```

```
student['Grade'] = 'A'
```

```
# Display the dictionary
```

```
print("Student Details:", student)
```

• **Output:**

```
Student Details: {'Name': 'Alice', 'Age': 21, 'Course': 'Computer Science', 'Grade': 'A'}
```

Aliasing

- When one dictionary variable is **assigned to another**, both refer to the **same memory location**.
- Any change made through one variable **affects the other**.

- **Example:**

```
dict1 = {'name': 'Alice', 'age': 25}
```

```
dict2 = dict1      # Aliasing
```

```
dict2['age'] = 30   # Modify using alias
```

```
print("dict1:", dict1)
```

```
print("dict2:", dict2)
```

- **Output:**

- dict1: {'name': 'Alice', 'age': 30}

- dict2: {'name': 'Alice', 'age': 30}

2. Copying

- To create a **separate copy** of a dictionary, use the **copy()** method.
- This creates a **shallow copy** — a new dictionary with the same key–value pairs.
- **Example:**

```
dict1 = {'name': 'Alice', 'age': 25}
```

```
dict3 = dict1.copy() # Copying
```

```
dict3['age'] = 30    # Modify the copy
```

```
print("dict1:", dict1)
```

```
print("dict3:", dict3)
```

- **Output:**
- dict1: {'name': 'Alice', 'age': 25}
- dict3: {'name': 'Alice', 'age': 30}

Check the output

```
dictionary = {"apples": 15, "bananas": 35, "grapes": 12}
```

```
>>> dictionary["bananas"]           Output:35
```

```
>>> dictionary["oranges"] = 20
```

```
Output: {'apples': 15, 'bananas': 35, 'grapes': 12, 'oranges': 20}
```

```
>>> len(dictionary)                 Output:4
```

```
>>> "grapes" in dictionary          Output: True
```

```
>>> dictionary["pears"]             Output: error
```

```
>>> dictionary.get("pears", 0)       Output:    0
```

-